

Spring & JPA Interview Notes

1. What is IoC (Inversion of Control) in Spring?

What: A design principle where object creation and dependency management is handed over to the Spring container instead of the code doing it itself. **Why:** Reduces tight coupling, makes code testable and flexible. **How:** Spring creates and wires objects (beans) for you via the `ApplicationContext`.

2. What is Dependency Injection (DI)?

What: A way of implementing IoC — dependencies are “injected” into a class instead of the class creating them. **Why:** Promotes loose coupling, easier unit testing (mock dependencies). **How:** Via constructor, setter, or field injection using annotations like `@Autowired`.

3. What is an IoC container?

What: The core Spring component that creates, configures, and manages bean lifecycle. **Why:** Centralizes object management so you don’t manually `new` objects. **How:** Implemented as `BeanFactory` (basic) or `ApplicationContext` (advanced, most used).

4. What is a Bean in Spring?

What: Any object that is created, managed, and injected by the Spring IoC container. **Why:** Standardizes how objects are created/reused across the app. **How:** Declared via `@Component`, `@Bean`, `@Service`, `@Repository`, or XML config.

5. What is Java-based configuration (`@Configuration` class)?

What: A Java class used to define beans instead of XML. **Why:** Type-safe, refactor-friendly, no XML clutter. **How:** Annotate class with `@Configuration`, define methods annotated `@Bean`.

6. Difference: `@Component`, `@Bean`, `@Service`, `@Repository`

Annotation	Used On	Purpose

<code>@Component</code>	Class	Generic Spring-managed bean
<code>@Service</code>	Class	Business/service layer (semantic <code>@Component</code>)
<code>@Repository</code>	Class	DAO layer; adds exception translation
<code>@Bean</code>	Method (inside <code>@Configuration</code>)	Manually define a bean, often for 3rd-party classes

Why separate: Improves readability and layered architecture, even though functionally similar.

7. What are Bean scopes in Spring?

What: Defines how many instances of a bean are created. **Types:**

- `singleton` (default) – one instance per container
- `prototype` – new instance every time requested
- `request` / `session` / `application` – web-specific scopes **Why:** Controls memory usage and state-sharing behavior.

8. What is the Bean lifecycle in Spring?

What: Stages a bean goes through: instantiate → populate properties →

`@PostConstruct` → ready for use → `@PreDestroy` → destroyed. **Why:** Lets you hook custom init/cleanup logic (e.g., open/close DB connection). **How:** Use `@PostConstruct` and `@PreDestroy`, or implement `InitializingBean` / `DisposableBean`.

9. What is `@Autowired` used for?

What: Automatically injects a required dependency (bean) into a class. **Why:** Removes need for manual object creation. **How:** Place on constructor (preferred), field, or setter. Spring resolves by type.

10. Types of Dependency Injection

1. **Constructor Injection** – via constructor params
2. **Setter Injection** – via setter methods
3. **Field Injection** – via `@Autowired` directly on field

11. Difference: constructor, setter, field injection

Type	Immutability	Testability	Circular Dep Support
Constructor	✅ Yes	✅ Best	❌ No
Setter	❌ No	✅ Good	✅ Yes
Field	❌ No	❌ Poor (needs reflection)	✅ Yes

12. Why is constructor injection preferred over field injection?

- Makes dependencies **immutable** (final fields)
- Ensures object is **fully initialized** at creation
- Easier to **unit test** (no need for Spring context/reflection)
- Fails fast if a dependency is missing

13. What is `@Qualifier` , and when is it used?

What: Specifies *which* bean to inject when multiple beans of the same type exist. **Why:**

`@Autowired` alone can't decide between multiple candidates → ambiguity error. **How:**

`@Qualifier("beanName")` next to `@Autowired`.

14. What is `@Primary` , and how does it differ from `@Qualifier` ?

What: Marks a bean as the **default choice** when multiple candidates exist. **Difference:**

- `@Primary` → set once on the bean definition (default winner)
- `@Qualifier` → set at injection point (explicit choice, overrides `@Primary`)

15. What is a circular dependency, and how does Spring handle it?

What: Bean A depends on Bean B, and B depends back on A. **Why it's a problem:** With constructor injection, Spring can't resolve it (chicken-egg) → throws exception. **How**

Spring handles it: Works with **setter/field injection** using early bean references (3-level cache). Best fix: **redesign** to remove the cycle.

16. What is component scanning (`@ComponentScan`)?

What: Tells Spring which packages to scan for annotated classes (`@Component` , `@Service` , etc.). **Why:** Auto-detects and registers beans without manual declaration.

How: `@ComponentScan(basePackages = "com.example")` , usually implied by

`@SpringBootApplication`.

17. What is `@Profile` used for?

What: Activates beans only for a specific environment (dev, test, prod). **Why:** Different configs for different environments (e.g., mock DB in test, real DB in prod). **How:**

`@Profile("dev")` on a bean/class; activate via `spring.profiles.active=dev`.

18. What is `@PropertySource` used for?

What: Loads values from an external `.properties` file into Spring's `Environment`. **Why:** Externalize configuration instead of hardcoding. **How:**

`@PropertySource("classpath:custom.properties")` + `@Value("${key}")`.

19. What is loose coupling, and why does Spring encourage it?

What: Classes depend on **abstractions (interfaces)**, not concrete implementations.

Why: Easier to swap implementations, test with mocks, and maintain code. **How Spring**

helps: DI + interfaces let you change implementation without touching dependent classes.

20. What is JPA (Java Persistence API)?

What: A **specification** (not implementation) for ORM — mapping Java objects to database tables. **Why:** Standardizes persistence code across different ORM providers.

How: Uses annotations like `@Entity`, `@Id`, `@Table`; implemented by Hibernate, EclipseLink, etc.

21. Difference between JPA and Hibernate

	JPA	Hibernate
Type	Specification (interface/rules)	Implementation (actual ORM engine)
Vendor	Part of Java EE/Jakarta	Provided by Red Hat
Usage	Defines <code>@Entity</code> , <code>EntityManager</code>	Implements them + extra features (caching, HQL)

Simple line: JPA = rulebook, Hibernate = one of the players following it.

22. What is a Repository in Spring Data, and why a marker interface?

What: An interface that abstracts data access (CRUD operations) without needing implementation code. **Why marker interface:** `Repository` itself has no methods — it just tells Spring "this is a data access component" so Spring can create a proxy

implementation automatically. **How:** Extend `JpaRepository / CrudRepository`; Spring generates implementation at runtime.

23. Difference: `CrudRepository`, `JpaRepository`, `PagingAndSortingRepository`

Interface	Extends	Adds
<code>CrudRepository</code>	—	Basic CRUD (save, findById, delete)
<code>PagingAndSortingRepository</code>	<code>CrudRepository</code>	Pagination & sorting
<code>JpaRepository</code>	<code>PagingAndSortingRepository</code>	JPA-specific: batch ops, flush, <code>getReferenceById</code>

Rule of thumb: Use `JpaRepository` — it has everything the others offer plus more.

24. What is the `@Transactional` annotation used for?

What: Wraps a method in a database transaction (commit on success, rollback on exception). **Why:** Ensures **atomicity** — all DB operations succeed together or none do.

How: Place on service method/class; Spring manages commit/rollback automatically via AOP proxy.

25. What is transaction propagation (`@Transactional(propagation = ...)`)?

What: Defines how a transaction behaves when one transactional method calls another.

Why: Controls whether calls join, suspend, or start a new transaction — important for nested service calls. **Common types:**

- `REQUIRED` (default) – join existing or create new
- `REQUIRES_NEW` – always start a new transaction, suspend current
- `NESTED` – runs within a savepoint of the current transaction
- `MANDATORY` – must run within existing transaction, else throws error